unyts

Unit Conversion with Directed Graph Networks

Powered by Graph Search Algorithms

Version 0.10.1

March 5, 2026

https://github.com/ayaranitram/unyts

Contents

Chapter 1

Introduction

`unyts` is an innovative Python package that revolutionizes unit conversion and unit-aware calculations by leveraging directed graph network theory combined with graph search algorithms.

1.1 The Core Innovation

Traditional unit converters rely on exhaustive lookup tables containing pre-computed conversion factors for every possible unit pair. This approach suffers from several limitations:

- Scalability: Adding new units requires defining conversions to all existing units
- Maintenance: Each new unit adds $O(n)$ conversion definitions
- Compound units: Ratios like `kg/m3` or `stb/day/psi` explode the lookup table size
- Memory overhead: Storing redundant conversion paths

The `unyts` package takes a fundamentally different approach by modeling units and conversions as a directed graph (digraph):

- Each unit is a node in the graph
- Each conversion rule is a directed edge with a transformation function
- Breadth-First Search (BFS) discovers conversion paths automatically
- Units are defined by their official definitions (e.g., 1 foot = 12 inches)

This graph-based architecture enables conversion between any two compatible units without requiring explicit definition of every conversion pair. The search algorithms find the optimal path through the network automatically.

1.2 Key Features

1.2.1 Pre-Loaded Unit Networks

The package ships with comprehensive networks covering:

- Length: SI (meter with prefixes) and Imperial (inch, foot, yard, mile, etc.)
- Area: square units for all length systems

- Volume: cubic units, gallons (US & UK), liters, barrels
- Mass/Weight: metric (gram, kilogram) and Imperial (pound, ounce, ton)
- Time: millisecond through century
- Temperature: Celsius, Fahrenheit, Kelvin, Rankine
- Pressure: Pascal, bar, psi (gauge & absolute)
- Energy, Power, Density, Viscosity, Data, and more

1.2.2 Unit-Aware Calculations

The `Unit` class hierarchy provides:

- Arithmetic operations: Addition, subtraction, multiplication, division with automatic unit handling
- Logical comparisons: Units automatically convert before comparison
- Exponentiation: Raising units to powers (e.g., m^2, ft^3)
- Compound unit creation: Multiplying `m` × `m` yields `m2`

1.2.3 Multiple Search Algorithms

Four specialized algorithms optimize different scenarios:

- BFS: Guarantees shortest path, layer-by-layer exploration
- lean_BFS: Optimized BFS with pre-filtered network (default)
- DFS: Fast depth-first search with descendant filtering
- hybrid_BFS: Parallel execution of BFS and lean_BFS

1.3 Use Cases

`unyts` excels in:

- Scientific computing: Heterogeneous unit handling in calculations
- Engineering applications: Converting between SI and Imperial systems
- Oil & gas industry: Specialized units (API gravity, standard barrels, formation volume factors)
- Data analysis: pandas/NumPy integration for unit-aware arrays
- Education: Teaching dimensional analysis and unit systems

1.4 Design Philosophy

1. Canonical definitions: Units defined by international standards (1 yard = 0.9144 meters)
2. No duplicate paths: Each unit pair connected by single canonical route
3. Composability: Complex ratios built from simple units
4. Extensibility: Add custom units without modifying existing network
5. Performance: Search memory caching for repeated conversions

The following chapters explore the technical foundations (graph theory, search algorithms), provide complete installation and usage guides, and document the full Python API.

Chapter 2

The Directed Graph Foundation

This chapter explains the graph-theoretic foundations of `unyts`, demonstrating why directed graphs provide superior unit conversion compared to traditional lookup tables.

2.1 Graph Theory Basics

2.1.1 What is a Directed Graph?

A directed graph (digraph) $G = (V, E)$ consists of:

- A set of vertices (nodes) $V = \{v_1, v_2, \ldots, v_n\}$
- A set of directed edges $E \subseteq V \times V$
- Each edge $e = (v_i, v_j)$ points from source v_i to target v_j

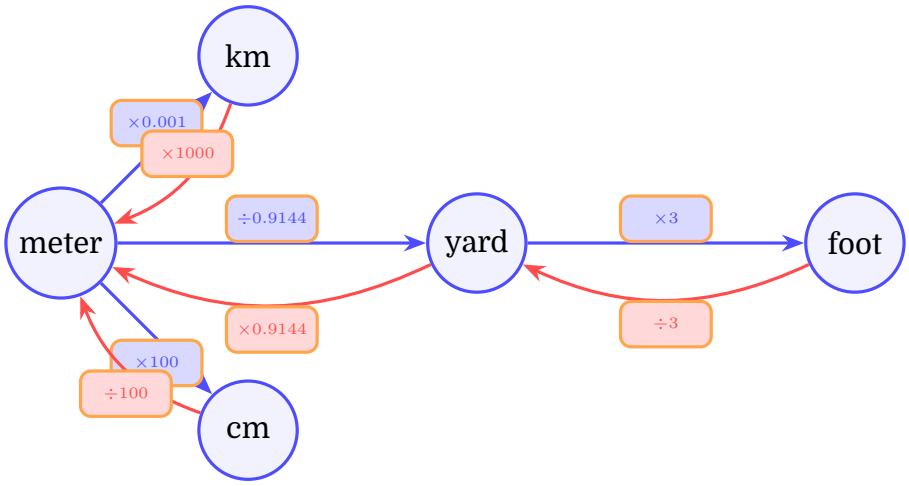

Figure 2.1: Simple directed graph showing length unit conversions. Circle nodes represent units. Rectangular boxes represent conversion factors. Blue edges represent forward conversions, red edges represent reverse conversions.

In Figure 2.1, each node represents a unit, and directed edges encode conversion factors. Notice that edges are bidirectional—both forward and reverse conversions exist.

Feature	Lookup Table	Graph Network
Add 1 unit	Define n conversions	Define $1-2$ edges
Storage	$O(n^2)$	$O(n)$ or $O(n \log n)$
Query time	$O(1)$	$O(n)$ with caching $\rightarrow O(1)$
Compound units	Exponential explosion	Compositional
Maintenance	Duplicate definitions	Canonical paths
Extensibility	Rigid	Dynamic

Table 2.1: Comparison of lookup tables vs. graph-based conversion

2.1.2 Graphs vs. Lookup Tables

The key advantage: adding a new unit to a graph requires connecting it to one or two existing units (e.g., connecting a new length unit only to `meter`), whereas a lookup table requires defining conversions to all existing units.

2.2 unyts Implementation

2.2.1 The UNode Class

Each unit in the network is represented by a `UNode` object:

```
class UNode:
    __slots__ = ('name',)

    def __init__(self, name: str):
        self.name = name

    def __eq__(self, other):
        return self.name == other.name

    def __hash__(self):
        return hash(self.name)
```

Listing 2.1: UNode implementation (simplified)

Key design choices:

- Equality based on name: Nodes created in different sessions compare equal
- Hashable: Enables use as dictionary keys for search memory caching
- Lightweight: Minimal memory footprint with `__slots__`

2.2.2 The Conversion Class

Directed edges encode transformation functions:

```
class Conversion:
    def __init__(self, source: UNode, target: UNode,
                 function: callable):
        self.source = source
        self.target = target
        self.function = function
```

Listing 2.2: Conversion class structure

A conversion from meters to kilometers stores:

```
meter_to_km = Conversion(
    source=UNode('meter'),
    target=UNode('kilometer'),
    function=lambda x: x * 0.001
)
```

2.2.3 The UDigraph Class

The master graph object manages the entire network:

```
class UDigraph:
    def __init__(self):
        self.edges = {}  # {UNode: ([children], [conversions])}
        self.memory = {} # Cached search results
        self.fvf = None  # Formation volume factor

    def add_node(self, node: UNode):
        if node not in self.edges:
            self.edges[node] = ([], [])

    def add_edge(self, conversion: Conversion):
        source = conversion.source
        if source not in self.edges:
            self.add_node(source)
        children, conversions = self.edges[source]
        children.append(conversion.target)
        conversions.append(conversion.function)

    def children_of(self, node: UNode):
        return self.edges[node][0] if node in self.edges else []
```

Listing 2.3: UDigraph core attributes

The `edges` dictionary maps each node to a tuple of `(child_nodes, conversion_functions)`, enabling efficient traversal during search.

2.2.4 Building the Network

Unit conversions are defined based on official standards:

```
network = UDigraph()

# Define nodes
meter = UNode('meter')
kilometer = UNode('kilometer')
yard = UNode('yard')
foot = UNode('foot')

network.add_node(meter)
network.add_node(kilometer)
network.add_node(yard)
network.add_node(foot)

# Define conversions (both directions)
network.add_edge(Conversion(meter, kilometer,
                            lambda x: x * 0.001))
```

```
network.add_edge(Conversion(kilometer, meter,
                            lambda x: x * 1000))
network.add_edge(Conversion(meter, yard,
                            lambda x: x / 0.9144))
network.add_edge(Conversion(yard, meter,
                            lambda x: x * 0.9144))
network.add_edge(Conversion(yard, foot,
                            lambda x: x * 3))
network.add_edge(Conversion(foot, yard,
                            lambda x: x / 3))
```

Listing 2.4: Example network construction

2.3 Why Graphs Win

2.3.1 Automatic Path Discovery

Once the network is built, conversions between any compatible unit pair are automatic. To convert 100 centimeters to feet:

1. BFS finds path: `cm` → `m` → `yard` → `foot`
2. Apply transformations: $100 \div 100 \times (1/0.9144) \times 3 = 3.281$ feet
3. Cache result for future use

   No explicit `cm` → `foot` conversion was ever defined!

2.3.2 Bidirectional Conversion

Every conversion automatically works in both directions. Defining `meter` → `yard` and its inverse enables conversions in either direction plus all transitive paths (meter → yard → foot).

2.3.3 Composability

Compound units emerge naturally:

- `kg/m3` decomposes to `kg` and `m3`
- Each component converts independently
- `stb/day/psi` = (standard barrels) / (day) / (pounds per square inch)

  The graph handles numerator and denominator units separately, then recombines.

2.3.4 Extensibility

Adding industry-specific units requires minimal changes:

```
from unyts import set_unit, set_conversion

# Add a custom unit
set_unit('my_unit')

# Define conversion to an existing unit
set_conversion('my_unit', 'meter',
```

```
                lambda x: x * 42.0,
                lambda x: x / 42.0)
```

Listing 2.5: Adding custom units

The new unit immediately participates in all graph search operations, enabling conversions to any compatible unit without additional definitions.

2.4 Summary

The directed graph architecture provides:

1. Scalability: $O(n)$ storage vs. $O(n^2)$ lookup tables
2. Maintainability: Canonical definitions prevent inconsistencies
3. Flexibility: Automatic discovery of conversion paths
4. Composability: Natural handling of compound units
5. Extensibility: Add units without modifying existing network

The next chapter explores the graph search algorithms that traverse this network efficiently.

Chapter 3

Search Algorithms

`unyts` implements four specialized graph search algorithms, each optimized for different conversion scenarios. This chapter explains their operation, performance characteristics, and use cases.

3.1 Breadth-First Search (BFS)

3.1.1 Algorithm Overview

BFS explores the graph layer by layer, visiting all nodes at distance d before exploring nodes at distance $d + 1$. This guarantees finding the shortest path between two nodes.

Key properties:

- Completeness: Finds a path if one exists
- Optimality: Always returns the shortest path
- Time complexity: $O(|V| + |E|)$ where V = nodes, E = edges
- Space complexity: $O(|V|)$ for the queue and visited set

3.1.2 Implementation

```
def BFS(graph, start, end):
    path_queue = [[start]]    # Queue of paths
    visited = []              # Visited paths

    while path_queue:
        current_path = path_queue.pop(0)  # Dequeue
        last_node = current_path[-1]

        if current_path in visited:
            continue
        visited.append(current_path)

        if last_node == end:
            return current_path  # Found!

        # Expand to children
        for child in graph.children_of(last_node):
            if child not in current_path:
                new_path = current_path + [child]
```

```
                path_queue.append(new_path)

    return None  # No path found
```

Listing 3.1: BFS pseudocode

3.1.3 Visual Example

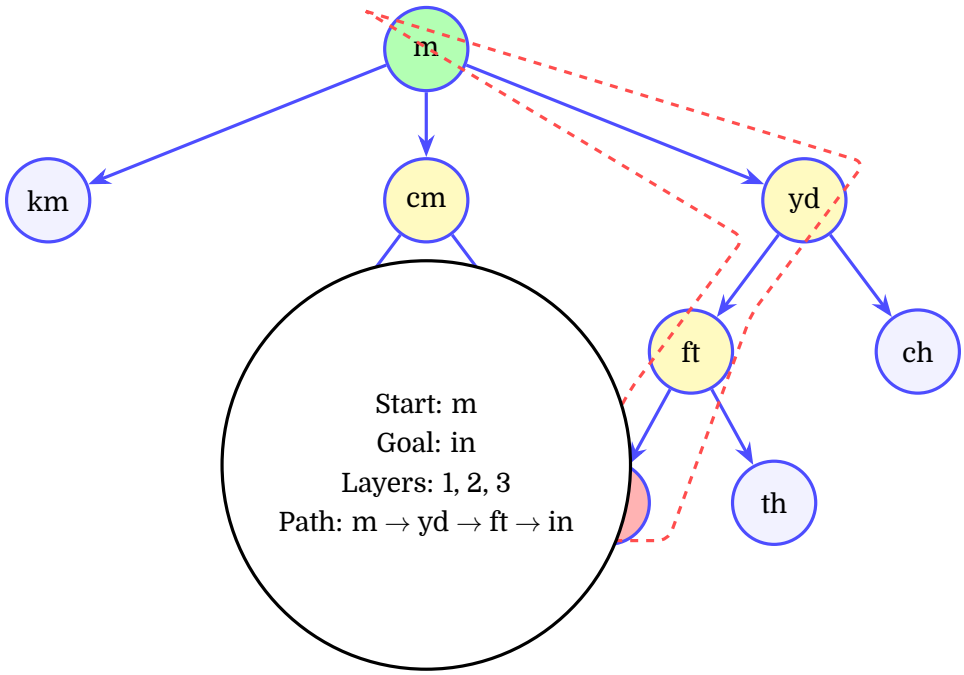

Figure 3.1: BFS exploration tree. Green node is start, yellow nodes explored before red goal node (in). Dashed red line shows shortest path found. All unit nodes are circles.

In Figure 3.1, BFS explores level 1 (km, cm, yd), then level 2 (mm, dm, ft, ch), discovering the shortest 3-hop path: meter → yard → foot → inch.

3.1.4 Use Cases

- When shortest conversion path is required
- Unknown network topology
- Verification of conversion accuracy
- Teaching/debugging conversion logic

3.2 Depth-First Search (DFS)

3.2.1 Algorithm Overview

DFS explores as deep as possible along each branch before backtracking. It uses descendant filtering to avoid exploring branches that cannot reach the goal.

Key properties:

- Completeness: Finds a path (but not necessarily shortest)

- Optimality: No guarantee of shortest path
- Time complexity: $O(|V| + |E|)$ with pruning
- Space complexity: $O(|V|)$ for recursion stack
- Advantage: Often faster for deep paths

3.2.2 Implementation with Descendant Filtering

The `unyts` DFS implementation adds intelligent pruning:

```python
def DFS(graph, start, end, branch_depth=25):
    def dfs_recursive(node, visited, path):
        visited.add(node)

        for child in graph.children_of(node):
            if child in visited:
                continue

            # Pruning: skip if goal unreachable within depth
            if end not in get_descendants(child, branch_depth):
                continue

            new_path = path + [child]

            if child == end:
                return new_path  # Found!

            result = dfs_recursive(child, visited, new_path)
            if end in result:
                return result

        return path

    return dfs_recursive(start, set(), [start])
```

Listing 3.2: DFS with descendant filtering

The `get_descendants(node, depth)` function precomputes all nodes reachable within `depth` hops, preventing exploration of dead-end branches.

3.2.3 Visual Example

DFS may explore fewer nodes than BFS if the goal happens to be on the first deep branch explored (though it might also explore more if unlucky).

3.2.4 Use Cases

- Known deep conversion paths (e.g., complex compound units)
- Memory-constrained environments
- When first valid path is acceptable

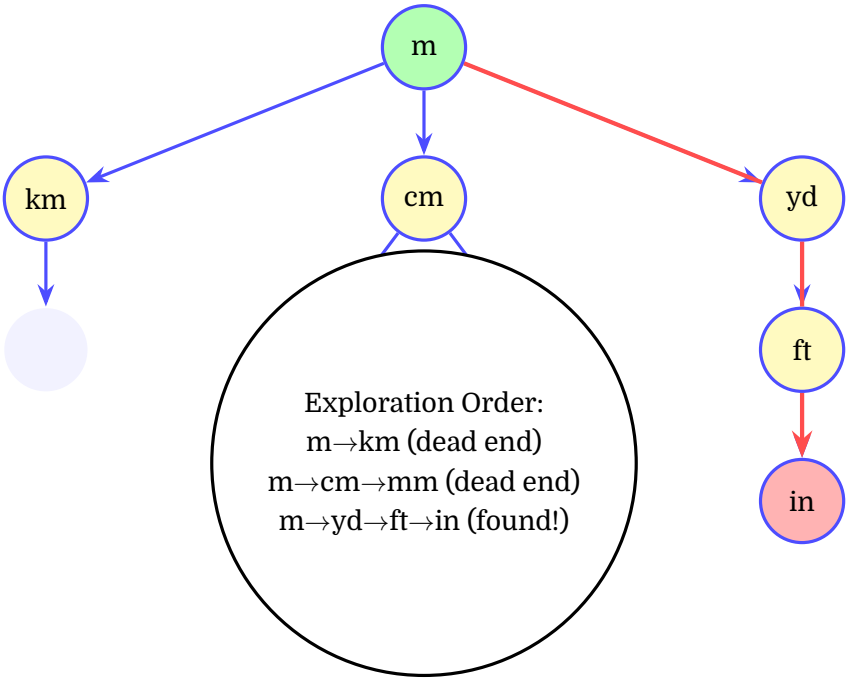

Figure 3.2: DFS exploration tree. Thick red arrows show the path DFS follows. Yellow nodes explored before finding goal. All unit nodes are circles.

3.3 lean_BFS (Optimized BFS)

3.3.1 Innovation: Network Pruning

The classic BFS bottleneck is exploring nodes irrelevant to the start→end conversion. lean_BFS addresses this by:

1. Compute descendants of `start` up to depth d: D_{start}
2. Compute descendants of `end` up to depth d: D_{end}
3. Find intersection: $D_{relevant} = D_{start} \cap D_{end}$
4. Construct slimmed network containing only nodes in $D_{relevant}$
5. Run standard BFS on this smaller network

3.3.2 Multi-Generation Strategy

lean_BFS iteratively tries increasing depths: $[0, 1, 2, 3, 4, 5, 7, 10, 13, 16, 20, 25, \ldots]$ until the intersection is non-empty. This avoids over-expanding for simple conversions while handling complex ones.

```
def lean_BFS(graph, start, end, max_gen=25):
    generations = [0, 1, 2, 3, 4, 5, 7, 10, 13, 16, 20, 25]

    for depth in generations:
        if depth > max_gen:
            break

        start_desc = get_descendants(start, depth)
        end_desc = get_descendants(end, depth)
```

```
        intersection = start_desc & end_desc

        if intersection:
            # Build slim network with only these nodes
            slim_edges = {
                node: graph.edges[node]
                for node in intersection
                if node in graph.edges
            }
            slim_graph = SlimUDigraph(slim_edges)
            return BFS(slim_graph, start, end)

    return None  # No path found
```

Listing 3.3: lean_BFS network filtering

3.3.3 Performance Gains

Typical performance improvement over standard BFS:

- Network size reduction: From 500+ nodes to 10–30 relevant nodes
- Speed improvement: 5–20× faster for typical conversions
- Memory savings: Proportional to network reduction

3.3.4 Use Cases

- Default algorithm for most conversions
- Production systems requiring fast response
- Repeated conversions between popular unit pairs

3.4 hybrid_BFS (Parallel Search)

3.4.1 Concurrent Execution

hybrid_BFS runs BFS and lean_BFS simultaneously in separate threads (or processes), returning the first successful result:

```
def hybrid_BFS(graph, start, end):
    results = {'bfs': '', 'lean_bfs': ''}

    thread_bfs = Thread(target=run_bfs,
                        args=(results, graph, start, end))
    thread_lean = Thread(target=run_lean_bfs,
                         args=(results, graph, start, end))

    thread_bfs.start()
    thread_lean.start()

    while True:
        if results['lean_bfs']:
            thread_bfs.terminate()
            return results['lean_bfs']
```

```
        elif results['bfs']:
            thread_lean.terminate()
            return results['bfs']
        elif results['bfs'] is None and results['lean_bfs'] is None:
            return None  # Both failed
```

Listing 3.4: hybrid_BFS parallel execution

3.4.2 Parallelization Options

`unyts` supports multiple parallel backends:

- Threading: Default for Python <3.13 (GIL-limited)
- Multiprocessing: Python 3.14+ with free-threading (true parallelism)
- Auto: Automatically selects best available method

3.4.3 Use Cases

- Unknown path complexity
- Time-critical conversions
- Multi-core systems
- When willing to trade CPU for speed

3.5 Algorithm Comparison

Algorithm	Shortest Path	Speed	Memory	Best For
BFS	Yes	Medium	High	Shortest guarantee
DFS	No	Fast (deep)	Low	Known deep paths
lean_BFS	Yes	Fast	Medium	Most conversions
hybrid_BFS	Yes	Fastest	Highest	Critical speed

Table 3.1: Search algorithm comparison

3.5.1 Selection Guidelines

- Default: Use `lean_BFS` (fast + optimal)
- Verification: Use `BFS` to confirm shortest path
- Complex units: Try `DFS` for deep compound ratios
- Maximum speed: Use `hybrid_BFS` with multiprocessing

The algorithm can be changed globally or per-conversion:

```
import unyts

# Set globally
unyts.set_algorithm('hybrid_BFS')

# All conversions use hybrid_BFS from now on
result = convert(100, 'cm', 'in')
```

3.6 Summary

unyts provides four specialized search algorithms, each optimized for different scenarios. The default lean_BFS combines the shortest-path guarantee of BFS with network pruning for excellent performance. Users can select algorithms based on their specific needs: verification (BFS), deep paths (DFS), or maximum speed (hybrid_BFS).

Chapter 4

The Units Network

This chapter explains how the unit conversion network is organized, how conversions are defined based on official standards, and visualizes key network segments.

4.1 Network Organization Principles

4.1.1 Canonical Definitions

Every unit conversion in `unyts` is based on official international standards or widely accepted definitions:

- 1 foot = 12 inches (by definition)
- 1 yard = 3 feet (by definition)
- 1 yard = 0.9144 meters (exact, 1959 international yard)
- 1 kilometer = 1000 meters (SI prefix definition)
- 1 pound = 0.45359237 kilograms (exact, by definition)

This ensures consistency and prevents accumulation of rounding errors across multiple conversions.

4.1.2 No Duplicate Paths

A key design principle: between any two units, there exists exactly one canonical conversion path. This prevents:

- Conflicting conversion factors
- Cyclic dependencies with rounding errors
- Ambiguity in path selection

For example, to convert `foot` to `meter`, the path is uniquely:

$$\text{foot} \to \text{yard} \to \text{meter}$$

There is no direct `foot` $\to$ `meter` edge, ensuring all foot-to-meter conversions use the same computation.

4.1.3 Modular Structure

The network is organized into loosely-coupled modules:

1. SI System: Each base unit (meter, gram, second) connects to its SI prefixes
2. Imperial System: Units form chains based on their definitions
3. Bridges: Connect SI and Imperial systems at specific units
4. Specialized Systems: Temperature, pressure gauges, oil field units

4.2 Length Network

4.2.1 SI Prefix Structure

The meter serves as the SI base unit for length, with prefix multipliers defined by powers of 10:

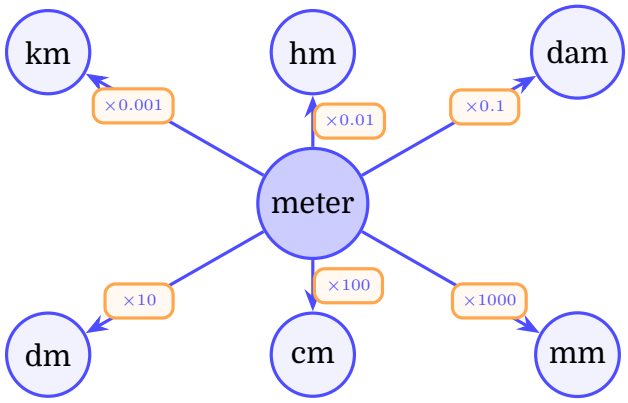

Figure 4.1: SI prefix network for length. Meter is the hub unit (blue center). Larger units above, smaller units below. All conversions route through the base unit. Circle nodes represent units, rectangular boxes represent conversion factors.

Additional SI prefixes (micro, nano, pico, kilo, mega, giga, etc.) follow the same pattern. The meter acts as a hub node for the SI subsystem.

4.2.2 Imperial Chain Structure

Imperial units form a linear chain based on their historical definitions:

4.2.3 SI-Imperial Bridge

The two systems connect at a single point: yard ↔ meter

4.2.4 Complete Length Network with Highlighted Path

Figure 4.4 shows the complete length network with a highlighted conversion path from centimeter to inch:

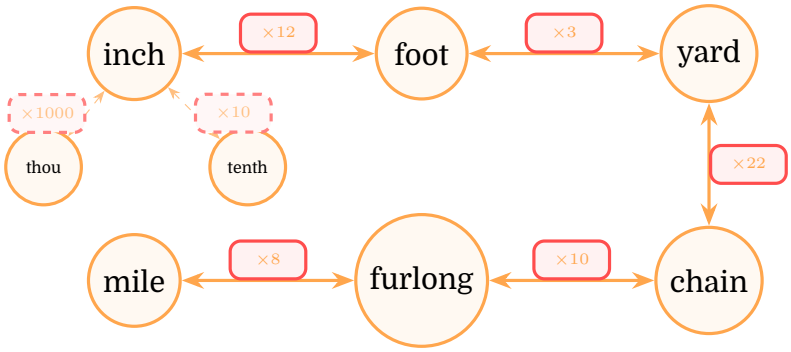

Figure 4.2: Imperial length chain (S-shaped layout). Top row flows left to right (inch→foot→yard), then wraps down to bottom row right to left (chain→furlong→mile). Circle nodes represent units, rectangular boxes represent conversion factors. Dashed lines show fractional sub-units.

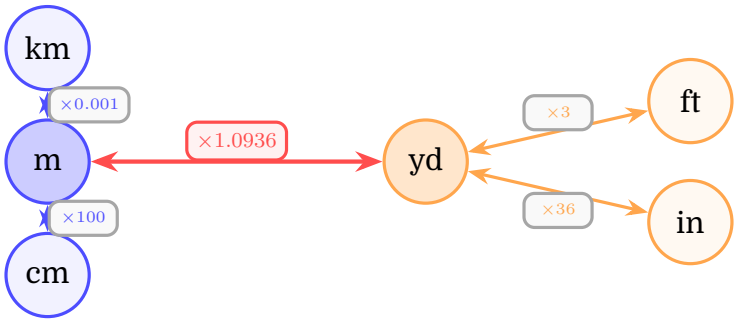

Figure 4.3: The SI-Imperial bridge. SI units (blue, left) connect to Imperial units (orange, right) through the meter↔yard bridge (red). All cross-system conversions must route through this connection. Circle nodes represent units, rectangular boxes represent conversion factors.

4.3 Time Network

The time network forms a hierarchical tree structure:

Note that `month` uses the average month length (30.4375 days) and conversions involving months are approximate. The year node connects to `day` (365.25 days) and `month` (12 months), providing both paths.

4.4 Temperature Network

Temperature conversions are non-linear due to offset + scale transformations:

The conversion functions for temperature include both multiplication and addition:

```
Celsius_to_Fahrenheit = lambda C: C * 9/5 + 32
Fahrenheit_to_Celsius = lambda F: (F - 32) * 5/9
```

Listing 4.1: Temperature conversion example

4.5 Other Network Segments

4.5.1 Volume Network

Volume conversions split into three subsystems:

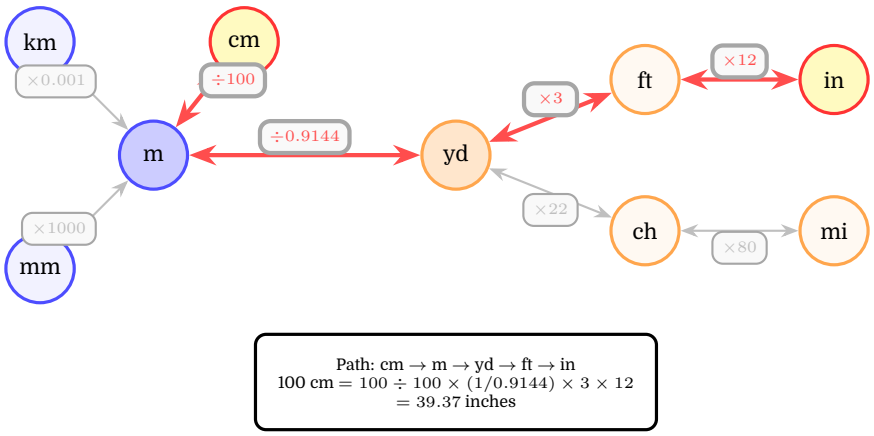

Figure 4.4: Complete length network showing highlighted conversion path from centimeter to inch (yellow nodes). Red thick arrows trace the conversion route. Gray connections show other available paths. Circle nodes represent units, rectangular boxes represent conversion factors.

- Cubic units: m^3, cm^3, ft^3, in^3 (derived from length)
- Liquid volume (US): gallon → quart → pint → gill → fluid ounce
- Liquid volume (UK): Imperial gallon → quart → pint (different definitions than US)
- Bridge: Liter ↔ cm^3 (exact), gallon(UK) ↔ liter, gallon(US) ↔ cubic inch

4.5.2 Pressure Network

Pressure includes both absolute and gauge measurements:

- Absolute: Pascal (Pa), bar, psi(a), atmosphere, Torr
- Gauge: barg ↔ bara (offset by atmospheric pressure), psig ↔ psia
- Conversions: bar ↔ Pascal ↔ atmosphere, psi ↔ bar

4.5.3 Mass Network

Similar to length, mass has SI (gram-based) and Imperial (pound-based) subsystems:

- SI: kilogram → gram (with prefixes: mg, kg, tonne)
- Imperial: pound → ounce → dram, ton → hundredweight → stone → quarter
- Bridge: pound ↔ kilogram (exact conversion)

4.6 Example Conversion Walkthrough

Let's trace the conversion of 100 centimeters to inches step by step:

4.6.1 Step 1: Find Path

lean_BFS searches and finds:

$$\texttt{cm} \to \texttt{m} \to \texttt{yard} \to \texttt{foot} \to \texttt{inch}$$

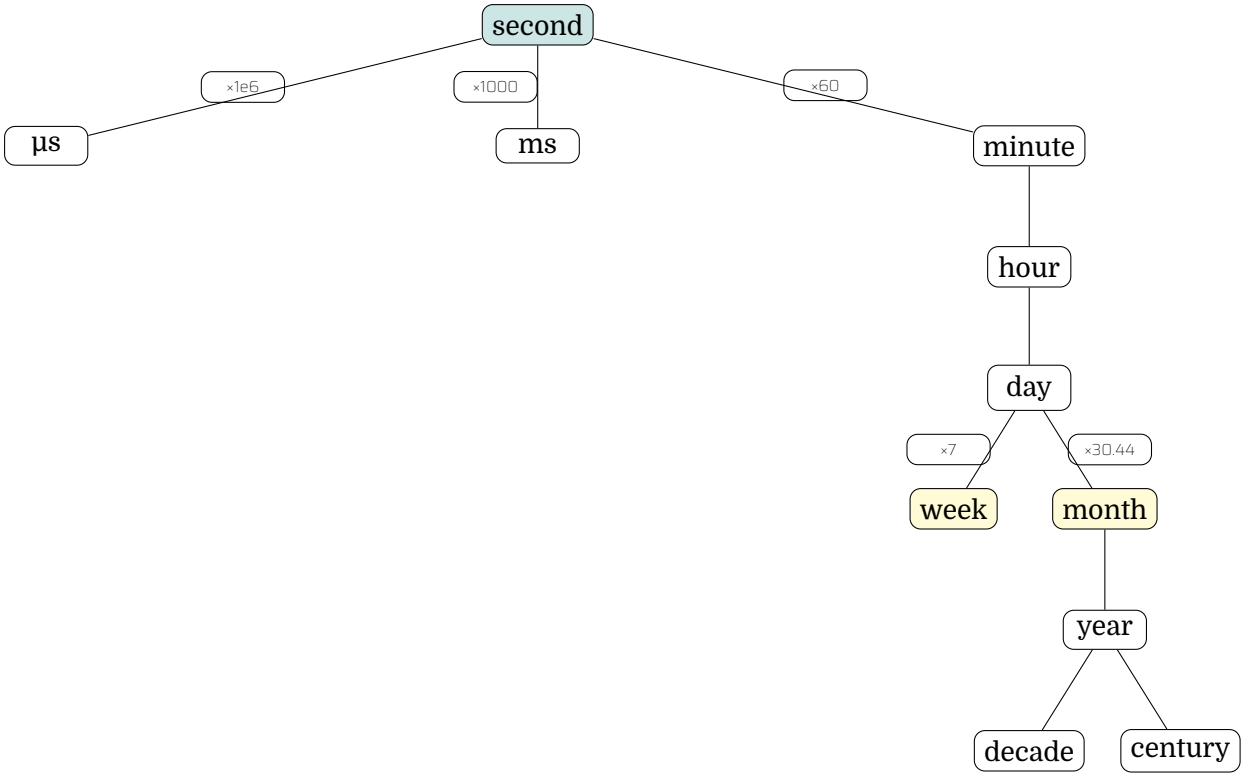

Figure 4.5: Time network hierarchical structure. Tree shows conversions from microseconds through seconds, minutes, hours, days to larger units. Yellow nodes (week, month) are alternatives branching from day.

4.6.2 Step 2: Apply Transformations

$$\begin{aligned}
\text{cm} \to \text{m}: &\quad 100 \div 100 = 1.0 \text{ meter} \\
\text{m} \to \text{yard}: &\quad 1.0 \div 0.9144 = 1.0936 \text{ yards} \\
\text{yard} \to \text{foot}: &\quad 1.0936 \times 3 = 3.2808 \text{ feet} \\
\text{foot} \to \text{inch}: &\quad 3.2808 \times 12 = 39.37 \text{ inches}
\end{aligned}$$

4.6.3 Step 3: Cache Result

The conversion path and final multiplier ($39.37/100 = 0.3937$) are cached:

```
units_network.memory[('cm', 'inch')] = {
    'path': ['cm', 'm', 'yard', 'foot', 'inch'],
    'factor': 0.3937
}
```

Future `cm` → `inch` conversions retrieve this cached result instantly ($O(1)$ lookup).

4.7 Summary

The `unyts` network is carefully organized around:

1. Official definitions: All conversions based on international standards
2. Single canonical paths: No duplicate routes between unit pairs

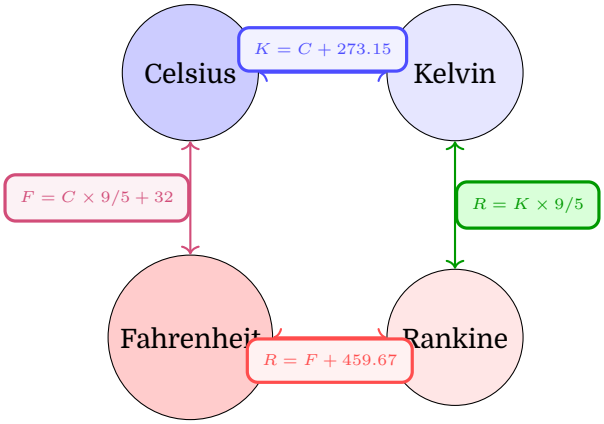

Figure 4.6: Temperature network showing all bidirectional conversions in rectangular boxes. Each conversion encodes both offset and scale transformations. Circle nodes represent temperature scales, rectangular boxes show conversion formulas.

3. Modular structure: SI, Imperial, and specialized subsystems
4. Strategic bridges: Minimal connections between subsystems
5. Hub topology: Base units (meter, gram, second) serve as central nodes

This architecture enables automatic discovery of conversion paths for thousands of unit combinations from a compact set of defining relationships.

Chapter 5

Installation

5.1 Requirements

5.1.1 Python Version

`unyts` supports Python 3.7 and later. The package metadata targets:

```
requires-python = ">=3.7"
```

Recommended: Python 3.10+ for optimal performance.

5.1.2 Core Dependencies

Required packages (automatically installed with `unyts`):

- `stringthings`: Helper utilities for string processing
- `cloudpickle`: Caching network and search memory

5.1.3 Optional Dependencies

While not required, the following packages enhance `unyts` functionality:

- NumPy: Enables array-aware conversions, handles lists/ndarrays
- Pandas: Recognizes Series/DataFrame objects for bulk conversions
- openpyxl: Required only for exporting the network to Excel format

Install optional dependencies manually if needed:

```
pip install numpy pandas openpyxl
```

5.2 Installation Methods

5.2.1 Install from PyPI (Recommended)

The simplest installation method:

```
pip install unyts
```

5.2.2 Upgrade to Latest Version

To ensure you have the latest features and bug fixes:

```
pip install --upgrade unyts
```

5.2.3 Install from Source

For development or testing unreleased features:

```
git clone https://github.com/ayaranitram/unyts.git
cd unyts
pip install -e .
```

The `-e` flag installs in editable mode, allowing you to modify source code and see changes immediately.

5.3 Verification

Verify the installation by checking the version:

```
import unyts
print(unyts.__version__)  # Should print: 0.10.1 (or later)
```

Test basic functionality:

```
from unyts import convert

result = convert(1, 'meter', 'foot')
print(result)  # Should print: 3.280839895013123
```

If no errors occur, `unyts` is correctly installed and operational.

5.4 First-Time Setup

On first import, `unyts` performs one-time initialization:

1. Builds the unit conversion network from definitions
2. Loads search memory cache (if available)
3. Creates user configuration directory

This process may take a few seconds. Subsequent imports are faster due to caching.

5.5 Configuration Files

`unyts` stores configuration and cache files in a user-specific directory:

- Windows: `C:\Users\<username>\.unyts\`
- macOS/Linux: `~/.unyts/`

Files stored:

- `parameters.ini`: User preferences (algorithm, print path, etc.)

- `search_memory.cache`: Cached conversion paths
- `units_network.cache`: Serialized network (future optimization)

To reset `unyts` to defaults, delete this directory and re-import.

5.6 Troubleshooting Installation

5.6.1 Missing cloudpickle

If you see warnings about cloudpickle:

```
Warning: Missing `cloudpickle` package.
Not able to cache search memory.
```

Install it explicitly:

```
pip install cloudpickle
```

5.6.2 Import Errors

If imports fail with `ModuleNotFoundError`, ensure `unyts` is installed in the active Python environment:

```
pip list | grep unyts
```

If using virtual environments or conda, activate the correct environment before installation.

5.6.3 Version Conflicts

If upgrading causes issues, completely remove the old version first:

```
pip uninstall unyts
pip install unyts
```

Chapter 6

Quick Start

This chapter provides a concise introduction to using `unyts` for common tasks.

6.1 Basic Conversion

The simplest way to use `unyts` is via the `convert()` function:

```
from unyts import convert

# Convert 10 feet to meters
meters = convert(10, 'ft', 'm')
print(meters)  # Output: 3.048
```

The function signature is:

```
convert(value, from_units, to_units,
        print_conversion_path=None, use_cache=True)
```

- `value`: Numeric input (int, float, list, np.array, pd.Series)
- `from_units`: Source unit string (e.g., `'ft'`, `'kg/m3'`)
- `to_units`: Target unit string
- `print_conversion_path`: Override global print_path setting
- `use_cache`: Set `False` to force fresh search

6.2 Creating Unit-Aware Quantities

Use the `units()` factory function to create `Unit` instances:

```
from unyts import units

q = units(6, 'in') + units(1, 'ft')
print(q)  # Output: 18_in
```

Arithmetic operations automatically handle unit conversions:

```
length = units(5, 'm')
width = units(200, 'cm')
area = length * width
print(area)  # Output: 10.0_m2
```

6.3 Type Checking

Check if a variable is unit-aware using `isinstance()`:

```
from unyts import Unit, units

value = units(5, 'm')
print(isinstance(value, Unit))  # Output: True
```

Important: Use `isinstance()`, not `type() is Unit`, because `units()` returns subclasses (e.g., `Length`, `Volume`).

6.4 Converting Unit Instances

Use the `.to()` method to convert instances:

```
length = units(100, 'cm')
print(length.to('in'))  # Output: 39.37_in
print(length.to('ft'))  # Output: 3.281_ft
```

6.5 Working with Arrays

`unyts` supports NumPy arrays and lists:

```
import numpy as np
from unyts import convert, units

# Convert list
temps_C = [0, 10, 20, 30]
temps_F = convert(temps_C, 'Celsius', 'Fahrenheit')
print(temps_F)  # [32.0, 50.0, 68.0, 86.0]

# Unit-aware array
distances = units(np.array([1, 2, 3, 4, 5]), 'km')
print(distances.to('mi'))
# Output: [0.621 1.243 1.864 2.485 3.107]_mi
```

6.6 Controlling Conversion Path Display

By default, `unyts` prints the conversion path on first use:

```
from unyts import convert
convert(1, 'km', 'mi')
# Output: km > m > yard > foot > inch > ... > mile
#         0.6213711922373339
```

Toggle this behavior globally:

```
import unyts

unyts.print_path(False)  # Disable
unyts.print_path(True)   # Enable
unyts.print_path()       # Toggle current state
```

Or control per-conversion:

```
convert(1, 'km', 'mi', print_conversion_path=True)
```

6.7 Changing the Search Algorithm

Switch algorithms globally:

```
import unyts

unyts.set_algorithm('BFS')          # Standard BFS
unyts.set_algorithm('lean_BFS')     # Optimized (default)
unyts.set_algorithm('DFS')          # Depth-first
unyts.set_algorithm('hybrid_BFS')   # Parallel
```

See Chapter 3 for algorithm details and selection guidelines.

Chapter 7

Graphical User Interface

The Unyts GUI provides an intuitive visual interface for unit conversions. Launch it from Python:

```
import unyts
unyts.start_gui()
```

Or from the command line:

```
python -m unyts
```

7.1 Main Window

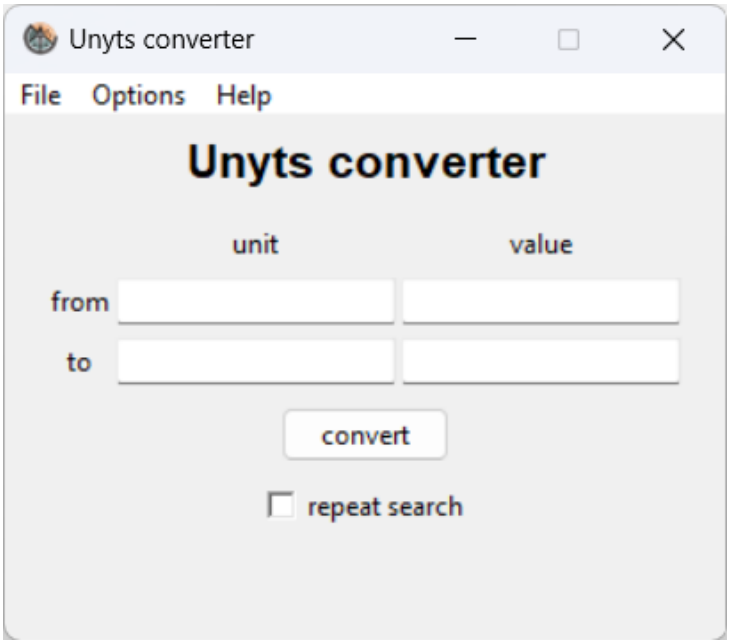

Figure 7.1: Unyts GUI main window showing conversion interface

The main window (Figure 7.1) features:

- From unit and From value: Source unit and quantity
- To unit and To value: Target unit and result

- Convert button: Execute conversion (or press Enter in any field)
- Repeat search checkbox: Force re-search (ignores cache)
- Conversion path display: Shows graph traversal path when enabled

7.2 Bidirectional Conversion

The GUI supports conversions in both directions:

- Forward: Enter value and unit in "from" fields, specify target "to" unit → press Enter or click Convert
- Reverse: Enter desired value in "to" field → calculates required "from" value

This bidirectional flexibility eliminates the need to invert calculations manually.

7.3 File Menu

7.3.1 Conversions Memory

The memory subsystem caches conversion factors to accelerate repeated conversions:

- Save memory: Persist cache to disk (happens automatically on exit if `Use cache` enabled)
- Clean memory: Clear all cached conversion factors from RAM
- Reload memory: Refresh cache from disk file
- Export memory: Save cache to custom file location
- Import memory: Load cache from external file

Use export/import to share conversion caches between machines or backup expensive search results.

7.3.2 Delete Cache Files

Removes all `.cache` files from the Unyts user directory. Useful for troubleshooting or forcing complete rebuild.

7.4 Options Menu

7.4.1 Set FVF (Formation Volume Factor)

For petroleum engineering workflows converting between reservoir and surface volumes. Enter FVF in:

- Dimensionless form: `1.25 vol/vol`
- With units: `1.5 rb/MMscf`

Once set, enables conversions like `convert(10, 'rb', 'sm3')` using FVF correction.

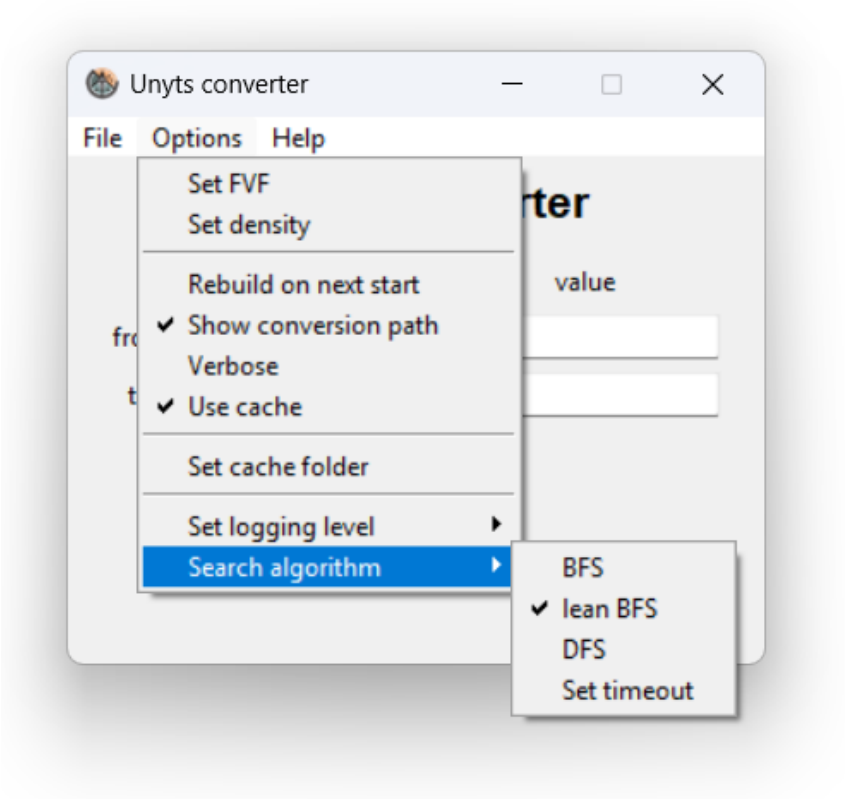

Figure 7.2: Options menu showing search algorithm selection

7.4.2 Set Density

Constant density for volume ↔ mass conversions. Enter in:

- Default: `997 g/cm³`
- With units: `8.34 lb/gal`

Enables conversions between incompatible dimensions, e.g., `convert(1, 'liter', 'kg')` using density bridge.

7.4.3 General Options

- Rebuild on next start: Force full network rebuild from database definitions (clears cache)
- Show conversion path: Display graph traversal path in marquee below button
- Verbose: Enable detailed console logging
- Use cache: Persist conversion factors across sessions
- Set cache folder: Change location where `.cache` files are stored
- Set logging level: Choose DEBUG, INFO, WARNING, ERROR, or CRITICAL verbosity

7.4.4 Search Algorithm

(Figure 7.2)
Select graph search algorithm:

- BFS: Standard breadth-first (guaranteed shortest path)
- lean BFS: Optimized BFS (default, 2.5× faster)
- hybrid BFS: Parallel BFS (experimental, requires threading/multiprocessing)
- DFS: Depth-first (faster for dense graphs, no guarantee of shortest path)
- Set timeout: Maximum search time in seconds (default 5, enter `unlimited` for no limit)

Changing algorithm automatically selects "Repeat search" to apply the new method.

7.4.5 Parallel

Enable experimental parallel search (requires Python threading or multiprocessing support):

- threading: Use Python threads (lighter weight)
- multiprocessing: Use separate processes (better CPU utilization)
- off: Disable parallelization
- auto: Automatic selection based on platform capabilities

Note: Parallel modes are currently experimental and deactivated by default on some platforms.

7.5 Help Menu

- Help: Opens Jupyter notebook demo at `https://github.com/ayaranitram/unyts/blob/master/unyts_demo.ipynb`
- GitHub: Links to repository at `https://github.com/ayaranitram/unyts`
- Version: Displays current Unyts version number

7.6 Conversion Path Display

When "Show conversion path" is enabled, a scrolling marquee below the button shows:

- Graph traversal path with conversion factors
- Error messages (e.g., "no conversion found", "search timeout")
- Status updates (e.g., "FVF set as 1.25 vol/vol")

Example path display:

```
[Unyts INFO]: converting from 'cm' to 'in': cm ×0.3937→ in
```

7.7 Error Handling in GUI

The GUI gracefully handles:

- NoConversionFoundError: Displays ”no conversion found” in path marquee
- SearchTimeoutError: Shows ”search timeout!” message
- NoFVFError: Prompts user to ”Please set FVF constant from Options menu”
- Invalid input: Non-numeric values are ignored until valid entry

Chapter 8

Python API Reference

The Unyts Python API provides functional and object-oriented interfaces for unit conversion, arithmetic, and manipulation.

8.1 Core Functions

8.1.1 convert()

Primary conversion function.
Signature:

```
convert(value, from_unit, to_unit, algorithm='lean_BFS',
        parallel=None, timeout=5, verbose=False)
```

Parameters:

- `value`: Numeric (int, float, array) to convert
- `from_unit`: Source unit as string (e.g., `'cm'`, `'kg/m3'`)
- `to_unit`: Target unit as string
- `algorithm`: `'BFS'`, `'lean_BFS'` (default), `'DFS'`, `'hybrid_BFS'`
- `parallel`: `None` (auto), `'t'` (threading), `'p'` (multiprocessing), `False`
- `timeout`: Maximum search time seconds (default 5)
- `verbose`: Boolean to enable logging

Returns: Converted numeric value
Raises:

- `NoConversionFoundError`: No path exists in graph
- `SearchTimeoutError`: Search exceeded timeout
- `NoFVFError`: FVF required but not set
- `WrongFormatError`: Invalid unit string format

Examples:

```python
from unyts import convert

# Basic conversion
convert(100, 'cm', 'm')  # → 1.0

# With NumPy array
import numpy as np
convert(np.array([1, 2, 3]), 'ft', 'in')  # → array([12, 24, 36])

# Complex units
convert(50, 'kg/m3', 'lb/ft3')  # → 3.121

# Algorithm selection
convert(1, 'mile', 'mm', algorithm='DFS')
```

8.1.2 units()

Factory function to instantiate appropriate Unit subclass.

Signature:

```python
units(value, unit_str, name=None)
```

Parameters:

- `value`: Numeric quantity
- `unit_str`: Unit string (determines subclass)
- `name`: Optional descriptive label

Returns: Instance of `Length`, `Mass`, `Time`, `Energy`, `Force`, or `Temperature` subclass

Examples:

```python
from unyts import units

length = units(50, 'cm')
print(length)  # → 50.0 cm
print(type(length).__name__)  # → Length

mass = units(10, 'kg', name='sample_weight')
print(mass.name)  # → sample_weight

temp = units(25, 'C')
print(type(temp).__name__)  # → Temperature
```

8.2 Unit Class

Base class for all unit objects. All subclasses (Length, Mass, Time, Energy, Force, Temperature, Unitless) inherit these methods.

8.2.1 Attributes

- `.value`: Numeric quantity (int, float, array)
- `.unit`: Unit string (e.g., `'m'`, `'kg/s2'`)

- `.name`: Optional descriptive string (default `None`)

Examples:

```
from unyts import units

distance = units(1500, 'm', name='track_length')
print(distance.value)  # → 1500
print(distance.unit)   # → m
print(distance.name)   # → track_length
```

8.2.2 Arithmetic Operations

Units support standard Python operators. Results adopt units of the first operand.

Addition / Subtraction

```
from unyts import units

a = units(100, 'cm')
b = units(1, 'm')
result = a + b
print(result)  # → 200.0 cm  (converts m to cm, then adds)

c = units(5, 'ft')
d = units(12, 'in')
print(c - d)   # → 4.0 ft  (converts in to ft, then subtracts)
```

Multiplication / Division

Creates compound units:

```
force = units(10, 'kg') * units(9.8, 'm/s2')
print(force)  # → 98.0 kg·m/s² (Force)

velocity = units(100, 'm') / units(10, 's')
print(velocity)  # → 10.0 m/s

# Dimensional analysis
energy = units(50, 'kg') * units(2, 'm/s')**2
print(energy)  # → 200.0 kg·m²/s² (Energy)
```

Exponentiation

```
area = units(5, 'm')**2
print(area)  # → 25.0 m²

volume = units(3, 'ft')**3
print(volume)  # → 27.0 ft³
```

8.2.3 Comparison Operators

Units automatically convert to common units before comparing:

```
from unyts import units

a = units(100, 'cm')
b = units(1, 'm')
print(a == b)    # → True (converts to same units)
print(a < b)     # → False
print(a <= b)    # → True

c = units(1, 'mile')
d = units(1600, 'm')
print(c > d)     # → True (1 mile = 1609.34 m)
```

8.2.4 .to() Method

Convert instance to different units, modifying in place:

```
from unyts import units

distance = units(1000, 'm')
distance.to('km')
print(distance)  # → 1.0 km
print(distance.value)  # → 1.0
print(distance.unit)   # → km

# Method chaining
temp = units(100, 'C').to('F')
print(temp)  # → 212.0 °F
```

8.3 Module-Level Functions

8.3.1 Configuration

```
import unyts

# Algorithm selection
unyts.set_algorithm('BFS')  # 'BFS', 'lean_BFS', 'DFS', 'hybrid_BFS'
current = unyts.get_algorithm()

# Parallelization
unyts.set_parallel('t')  # 't' (threading), 'p' (multiprocessing), False, None
unyts.set_parallel(False)  # Disable

# Search timeout
unyts.set_timeout(10)  # seconds, None for unlimited

# Logging
unyts.verbose(True)   # Enable verbose output
unyts.verbose(False)  # Disable

# Network rebuild
unyts.reload()  # Force rebuild from database
```

8.3.2 Memory Management

```
from unyts.database import save_memory, load_memory, clean_memory

# Cache operations
save_memory()           # Persist to default location
save_memory('custom.cache')  # Export to file
load_memory()           # Reload from default
load_memory('custom.cache')  # Import from file
clean_memory()          # Clear cache from RAM
```

8.3.3 Path Inspection

Visualize conversion paths:

```
import unyts

unyts.print_path(True)   # Enable path printing
result = unyts.convert(100, 'cm', 'in')
# Output: [Unyts INFO]: converting from 'cm' to 'in':
#         cm ×0→.3937 in

unyts.print_path(False)  # Disable
```

8.3.4 Custom Units and Conversions

Add New Unit

```
from unyts.database import units_network

# Define unit with conversion to existing unit
units_network.set_unit('smoot', 1.7018, 'm')
# 1 smoot = 1.7018 meters

result = unyts.convert(10, 'smoot', 'ft')
# Now 'smoot' is available in all conversions
```

Add Direct Conversion

```
from unyts.database import units_network

# Add conversion factor between two existing units
units_network.set_conversion('furlong', 'km', 0.201168)
# Sets: 1 furlong = 0.201168 km

# Creates bidirectional edge in graph
result = unyts.convert(5, 'km', 'furlong')
```

8.3.5 Formation Volume Factor (FVF)

For petroleum engineering:

```
from unyts.database import units_network

# Set FVF (reservoir vol / surface vol)
units_network.set_fvf(1.25)  # dimensionless vol/vol

# Now enables conversions:
result = unyts.convert(1000, 'rb', 'sm3')  # reservoir barrel → surface m³
```

8.3.6 Constant Density

Bridge volume and mass:

```
import unyts

# Set density for volume ↔ mass conversions
unyts.set_density(997)  # g/cm³ (water at 25°C)

# Now enables:
mass = unyts.convert(1, 'liter', 'kg')  # → 0.997 kg
volume = unyts.convert(500, 'g', 'ml')  # → ~500.5 ml
```

8.4 Type Checking and Subclasses

```
from unyts import units
from unyts.units import Length, Mass, Temperature

distance = units(100, 'm')
weight = units(50, 'kg')

# Type checking
isinstance(distance, Length)    # → True
isinstance(weight, Mass)        # → True
isinstance(weight, Length)      # → False

# Direct instantiation (alternative to units())
temp = Temperature(25, 'C')
force = Force(100, 'N')
```

8.5 NumPy and Pandas Integration

Unyts seamlessly handles array-like values:

```
import numpy as np
import pandas as pd
from unyts import convert, units

# NumPy arrays
arr = np.array([1, 2, 3, 4, 5])
result = convert(arr, 'km', 'm')
print(result)  # → [1000. 2000. 3000. 4000. 5000.]

# Pandas Series
```

```
series = pd.Series([10, 20, 30], name='distances')
converted = convert(series, 'mile', 'km')
print(converted)
# 0   16.09
# 1   32.19
# 2   48.28
# Name: distances, dtype: float64

# Unit objects with arrays
temps = units(np.array([0, 25, 100]), 'C')
temps.to('F')
print(temps.value)  # → [ 32.  77. 212.]
```

8.6 Uncertain Units

For measurements with uncertainty:

```
from unyts import units

# Create uncertain unit
length = units("100±5", "cm", name="measurement")
print(length)  # → (100±5) cm

# Propagates through arithmetic
doubled = length * 2
print(doubled)  # → (200±10) cm

# Conversion preserves uncertainty
in_meters = length.to('m')
print(in_meters)  # → (1.0±0.05) m
```

Chapter 9

Troubleshooting

9.1 Installation Issues

9.1.1 Module Not Found

Problem: `ModuleNotFoundError: No module named 'unyts'`

Solution:

1. Verify installation: `pip list | grep unyts`
2. Reinstall: `pip install --upgrade --force-reinstall unyts`
3. Check Python environment (virtual env vs system)
4. Ensure pip installs to correct Python interpreter

9.1.2 Dependency Conflicts

Problem: Package version conflicts during installation

Solution:

```
# Create clean virtual environment
python -m venv unyts_env
source unyts_env/bin/activate  # Linux/Mac
unyts_env\Scripts\activate     # Windows

pip install unyts
```

9.2 Conversion Errors

9.2.1 NoConversionFoundError

Problem: `unyts.errors.NoConversionFoundError: No conversion found between 'X' and 'Y'`

Causes and Solutions:

1. Incompatible dimensions: Cannot convert length to mass without density

```
# Won't work: different dimensions
convert(10, 'kg', 'm')  # ✗ Mass ≠ Length

# Solution: set density bridge
unyts.set_density(1000)  # kg/m³
convert(10, 'kg', 'liter')  # ✓ Mass → Volume via density
```

2. Typo in unit name:

```
convert(5, 'kilogram', 'g')  # ✗ Use 'kg' not 'kilogram'
convert(5, 'kg', 'g')        # ✓ Correct abbreviation
```

3. Custom unit not defined:

```
# Define custom unit first
from unyts.database import units_network
units_network.set_unit('smoot', 1.7018, 'm')
convert(10, 'smoot', 'ft')  # ✓ Now works
```

4. Disconnected graph component: Unit exists but has no path to target

```
# Add conversion bridge
units_network.set_conversion('unit_a', 'unit_b', factor)
```

9.2.2 SearchTimeoutError

Problem: `unyts.errors.SearchTimeoutError: Search exceeded timeout limit`
Solutions:

1. Increase timeout:

```
unyts.set_timeout(30)  # 30 seconds
# or
convert(value, from_unit, to_unit, timeout=30)
```

2. Disable timeout:

```
unyts.set_timeout(None)  # Unlimited time
```

3. Try faster algorithm:

```
unyts.set_algorithm('DFS')  # Often faster for complex graphs
```

4. Enable caching: Cache results to avoid repeated searches

```
unyts.cache(True)
convert(value, from_unit, to_unit)  # First call: slow search
convert(value2, from_unit, to_unit) # Second call: instant (cached)
```

9.2.3 NoFVFError

Problem: `unyts.errors.NoFVFError: FVF must be set for reservoir/surface volume conversions`
Solution:

```
from unyts.database import units_network

# Set formation volume factor
units_network.set_fvf(1.25)  # vol/vol

# Now works:
convert(1000, 'rb', 'sm3')  # Reservoir barrel → surface m³
```

9.2.4 WrongFormatError

Problem: `unyts.errors.WrongFormatError: Invalid unit format`
Common causes:

- Spaces in compound units: use `'kg/m3'` not `'kg / m3'`
- Unicode issues: ensure UTF-8 encoding for symbols like `'°C'`
- Unsupported operators: use `/` or `·` for division/multiplication

9.3 GUI Issues

9.3.1 GUI Won't Launch

Problem: `python -m unyts` fails or hangs
Solutions:

1. Check Tkinter installed:

```
python -c "import tkinter"  # Should not error
```

2. On Linux, install Tkinter:

```
# Ubuntu/Debian
sudo apt-get install python3-tk

# Fedora/RHEL
sudo dnf install python3-tkinter
```

3. Verify display server (Linux):

```
echo $DISPLAY  # Should output :0 or similar
```

9.3.2 GUI Crashes on Conversion

Problem: GUI closes or freezes during conversion
Solutions:

- Disable parallel mode: Options → Parallel → off
- Reduce timeout: Options → Search algorithm → Set timeout → 5 seconds
- Clear cache: File → Delete cache files, then restart
- Check console for error messages (run from terminal)

9.4 Performance Issues

9.4.1 Slow First Conversion

Cause: First run builds units network from database definitions (typically 2-5 seconds)
Solutions:

- Enable caching: `unyts.cache(True)` — subsequent runs load from cache instantly
- Pre-build cache: Run `python -m unyts` once to generate cache
- Cache persists across sessions in user folder

9.4.2 Slow Subsequent Conversions

Causes and solutions:

1. Complex path: Many graph hops required

```
# Add direct conversion shortcut
from unyts.database import units_network
units_network.set_conversion('unit_a', 'unit_b', factor)
```

2. Inefficient algorithm:

```
unyts.set_algorithm('lean_BFS')  # Fastest for most cases
# or
unyts.set_algorithm('DFS')       # Fast for dense graphs
```

3. Cache disabled:

```
unyts.cache(True)  # Enable persistent caching
```

9.5 Cache Corruption

Symptoms: Incorrect conversion results, crashes, or "file corrupt" errors
Solution:

```
from unyts.database import delete_cache
delete_cache()  # Remove all .cache files

# Or manually delete from:
# Windows: C:\Users\<user>\AppData\Local\unyts\
# Linux/Mac: ~/.unyts/

unyts.reload()  # Force rebuild
```

9.6 Logging and Debugging

9.6.1 Enable Verbose Mode

See detailed search paths and timing:

```
unyts.verbose(True)
convert(100, 'cm', 'in')
# Output:
# [Unyts INFO]: converting from 'cm' to 'in': cm ×0→.3937 in
# [Unyts DEBUG]: Search completed in 0.0012s
```

9.6.2 Adjust Logging Level

```
from unyts.helpers.logger import logger

logger.set_level('DEBUG')   # Most verbose
logger.set_level('INFO')    # Default
logger.set_level('WARNING') # Errors only
```

9.6.3 Inspect Network Structure

```python
from unyts.database import units_network

# View all units
all_units = units_network.get_all_units()
print(f"Total units: {len(all_units)}")

# Check if unit exists
if 'smoot' in all_units:
    print("Unit 'smoot' is defined")

# View conversion graph statistics
graph = units_network.get_graph()
print(f"Nodes: {len(graph.nodes)}, Edges: {len(graph.edges)}")
```

9.7 Getting Help

- GitHub Issues: https://github.com/ayaranitram/unyts/issues
- Demo Notebook: https://github.com/ayaranitram/unyts/blob/master/unyts_demo.ipynb
- Documentation: Browse source code docstrings for detailed API reference
- Email: Contact maintainer Martín Carlos Araya at martinaraya@gmail.com

When reporting issues, include:

1. Unyts version: `python -c "import unyts; print(unyts.__version__)"`
2. Python version: `python --version`
3. Operating system
4. Minimal code example reproducing the problem
5. Full error traceback

Appendix A

Comprehensive Unit Reference

This appendix lists all recognized units in Unyts, grouped by physical quantity type and sorted alphabetically within each category.

A.1 Length Units

SI Metric: angstrom, centimeter (cm), decimeter (dm), dekameter (dam), hectometer (hm), kilometer (km), meter (m), micrometer (µm), millimeter (mm), nanometer (nm), picometer (pm)

Imperial: chain (ch), fathom, foot (ft), furlong, inch (in), league, mile (mi), nautical mile, rod, thou, yard (yd)

Specialized: astronomical unit (AU), light-year (ly), parsec, point, pica

A.2 Mass Units

SI Metric: gram (g), kilogram (kg), microgram (µg), milligram (mg), nanogram (ng), tonne, metric ton

Imperial: carat, dram, grain, ounce (oz), penny weight, pound (lb), pound-force (lbf), stone, ton (US), ton (UK)

Atomic: atomic mass unit (u, amu)

A.3 Time Units

SI Metric: attosecond, centisecond, day, decisecond, femtosecond, hour (h), microsecond (µs), millisecond (ms), minute (min), nanosecond (ns), picosecond (ps), second (s)

Calendar: century, decade, lustrum, month (mo), week (w, we), year (y, yr)

A.4 Temperature Units

Absolute: Kelvin (K), Rankine (R)

Relative: Celsius (C, °C), Fahrenheit (F, °F)

Temperature Gradients: K/m, °C/m, °C/100m, °F/100ft, °F/ft

A.5 Volume Units

SI Metric: cubic centimeter (cm^3, cc, ml), cubic decimeter (dm^3), cubic meter (m^3), liter (L, l)

Imperial (US): barrel (bbl), bushel, cubic foot (ft^3), cubic inch (in^3), cubic yard (yd^3), cup, fluid ounce (fl oz), gallon (gal), gill, pint (pt), quart (qt), teaspoon (tsp), tablespoon (tbsp)

Imperial (UK): imperial barrel, imperial bushel, imperial cup, imperial fluid ounce, imperial gallon, imperial pint, imperial quart

Oil Industry: stock tank barrel (stb), reservoir barrel (rb)

A.6 Pressure Units

SI Metric: bar, millibar (mbar), pascal (Pa), kilopascal (kPa), megapascal (MPa)

Imperial: atmosphere (atm), psi (pound per square inch), psf (pound per square foot)

Other: mmHg (millimeter of mercury), torr, $dyne/cm^2$

A.7 Pressure Gradient Units

Common petroleum measurements: psi/foot, psi/meter, bar/foot, bar/meter, Pa/meter

A.8 Area Units

SI Metric: square centimeter (cm^2), square decimeter (dm^2), hectare (ha), square kilometer (km^2), square meter (m^2), square millimeter (mm^2)

Imperial: acre, square chain, square foot (ft^2), square inch (in^2), square mile (mi^2), square rod, square yard (yd^2)

A.9 Density Units

SI/Metric: g/cm^3, g/ml, kg/m^3, kg/L, metric ton/m^3

Imperial: lb/ft^3, lb/gallon (lb/gal), lb/in^3

Oil industry: API degrees (°API)

A.10 Velocity / Speed Units

SI Metric: centimeter per second (cm/s), kilometer per hour (km/h), meter per second (m/s)

Imperial: foot per second (ft/s), mile per hour (mph), knot (nautical mile per hour)

A.11 Rate Units

Volume flow: barrel per day (bbl/day), cubic meter per second (m^3/s), cubic meter per hour (m^3/h), liter per second (L/s), gallon per minute (gpm)

Mass flow: kilogram per second (kg/s), metric ton per hour (ton/h)

A.12 Force Units

SI Metric: dyne, kilonewton (kN), millinewton (mN), newton (N)

Imperial: kilogram-force (kgf), pound-force (lbf), poundal

A.13 Energy / Work / Heat Units

SI Metric: calorie (cal), foot-pound, joule (J), kilocalorie (kcal), kilowatt-hour (kWh), megajoule (MJ), millijoule (mJ), watt-hour (Wh)
Imperial: British thermal unit (BTU)
Energy equivalent: barrel of oil equivalent (boe), ton of TNT

A.14 Power Units

SI Metric: kilowatt (kW), megawatt (MW), milliwatt (mW), watt (W)
Imperial: horsepower (hp), foot-pound per second (ft-lbf/s)

A.15 Viscosity Units

Dynamic: centipoise (cP), poise, pascal-second (Pa·s)
Kinematic: centistoke (cSt), stoke (St), square meter per second (m^2/s)

A.16 Permeability Units

Darcy system: darcy, millidarcy (mD), microdarcy (µD)
SI system: square meter, square micrometer

A.17 Data Units

Bytes: byte (B), kilobyte (KB), megabyte (MB), gigabyte (GB), terabyte (TB), petabyte (PB)
Bits: bit (b), kilobit (kb), megabit (Mb), gigabit (Gb)

A.18 Dimensionless Quantities

Ratios and factors: percent (%), parts per million (ppm), parts per billion (ppb)
Petroleum-specific: API degree (°API, dimensionless measure of crude oil gravity)

A.19 Notes on Unit Selection

A.19.1 SI vs. Imperial Choice

- SI (metric) units are recommended for scientific applications and are the international standard
- Imperial units are common in aerospace, petroleum, and US-based engineering
- Unyts supports both seamlessly—use whichever is appropriate for your application
- When mixing systems, conversions route through defined bridges (e.g., meter ↔ yard for length)

A.19.2 Petroleum Abbreviations

Special note to petroleum engineers:

- bbl: Stock tank barrel (one standard definition)
- rb: Reservoir barrels (requires FVF setting for conversion)
- sm3: Surface cubic meters (stock tank equivalent)
- rm3: Reservoir cubic meters
- Ksm3: Thousand surface cubic meters
- BBL/day: Barrels per day (rate unit)
- mD: Millidarcy (permeability, crucial for flow calculations)

A.19.3 Temperature Notes

Temperature conversions are more complex than simple unit conversions:

- Absolute scales (Kelvin, Rankine): Can be converted directly
- Relative scales (Celsius, Fahrenheit): Have offsets; conversion is not just a factor
- Temperature differences/gradients: Always use Kelvin or Rankine intervals; °C/K differences are equivalent

Example:

```
# Correct: Temperature value (includes offset)
unyts.convert(0, 'C', 'F')  # → 32 °F (freezing point)

# Incorrect: Treating Celsius difference as temperature
# A 100°C temperature range = 180°F range (different!)
# For differences, use Kelvin intervals

# Correct: Temperature difference
# A 100 K difference = 180 R difference
unyts.convert(100, 'K/s', 'R/s')  # Rate of temperature change
```

A.20 Extending the Unit Database

To add custom units or new unit definitions to your Unyts installation:

```
from unyts.database import units_network

# Define a new unit relative to an existing one
units_network.set_unit('smoots', 5.1829, 'foot')
# Now 'smoot' is recognized in all conversions

# Add direct conversion between units
units_network.set_conversion('furlong', 'hectometer', 0.20117)

# Persist for next session
units_network.set_fvf(1.25)      # Petroleum FVF
unyts.set_density(850)           # Custom density in kg/m³
```

See Chapter 8 (Python API Reference), Section 8.6 for detailed API documentation.